

Documentation de gestion de projet - Vite & Gourmand

Projet ECF - TP Développeur Web et Web Mobile (Studi)

Auteur : Florian Marques

Ce document présente la méthode de gestion de projet que j'ai suivie pour réaliser l'application "Vite & Gourmand" : organisation du travail, backlog, découpage en étapes, workflow Git et suivi.

1. Méthode adoptée

J'ai gardé une organisation simple. J'ai monté un tableau Trello au début pour poser le backlog et avoir une vue d'ensemble, mais dans les faits j'ai surtout suivi mon travail directement sur le dépôt Git, branche par branche. Le Trello m'a servi de repère et je l'ai remis à jour à la fin. Je suis conscient qu'en équipe un outil comme Trello devient presque indispensable pour se répartir et coordonner le travail, ce qui n'était pas mon cas ici. Pour moi, c'est resté un suivi visuel léger, sans réunions ni rôles d'équipe.

Trois principes ont structuré le travail :

- **Un tableau visuel pour ne rien perdre de vue.** Le tableau Trello (colonnes Backlog, À faire, En cours, En test, Terminé) permet de voir d'un coup d'oeil où en est chaque tâche.
- **Une fonctionnalité = une branche = quelque chose de testable.** Chaque carte du backlog correspond à une branche Git et produit quelque chose de vérifiable une fois mergé. C'est la règle qui a le plus cadré le rythme (voir la section Git).
- **Le MVP d'abord, les détails ensuite.** J'ai traité les fonctionnalités obligatoires du cahier des charges avant tout confort ou amélioration. La priorisation MoSCoW (ci-dessous) m'a servi de garde-fou.

Au début de l'ECF, j'ai vite compris qu'une appli web complète, ça fait un gros projet pour une seule personne. J'ai donc pris le temps dès le départ de bien réfléchir à comment le découper en étapes indépendantes pour arriver au résultat que je présente. L'estimation indicative du sujet est de 70 heures. En vrai, le travail s'est étalé sur plusieurs semaines, avec des périodes de code intenses et d'autres où j'avais moins vite sur le code parce que je faisais surtout de la veille, des recherches et des tests en local pour trouver les bonnes directions et faire les choix les plus logiques. Le découpage en étapes indépendantes m'a beaucoup aidé à reprendre le fil quand je revenais dessus.

2. Backlog produit

Le backlog est exprimé en user stories, groupées par acteur et priorisées selon la méthode MoSCoW (Must / Should / Could / Won't). Les priorités reflètent le cahier des charges : tout ce que le sujet exige est en "Must".

2.1 Visiteur (public)

#	User story	Priorité
V1	En tant que visiteur, je veux consulter la page d'accueil (présentation, équipe, avis) pour découvrir l'entreprise.	Must
V2	En tant que visiteur, je veux voir la liste des menus pour comparer les offres.	Must
V3	En tant que visiteur, je veux filtrer les menus (prix, thème, régime, nombre de personnes) sans rechargement pour affiner ma recherche.	Must
V4	En tant que visiteur, je veux voir le détail d'un menu pour connaître les plats, allergènes et conditions.	Must
V5	En tant que visiteur, je veux envoyer un message via le formulaire de contact pour poser une question.	Must
V6	En tant que visiteur, je veux créer un compte pour pouvoir commander.	Must
V7	En tant que visiteur, je veux consulter les mentions légales et les CGV pour être informé.	Should

2.2 Utilisateur (connecté)

#	User story	Priorité
U1	En tant qu'utilisateur, je veux me connecter et me déconnecter pour accéder à mon espace.	Must
U2	En tant qu'utilisateur, je veux réinitialiser mon mot de passe oublié pour récupérer mon accès.	Must
U3	En tant qu'utilisateur, je veux commander un menu avec calcul du prix (proratisation, réduction, livraison) pour passer commande en ligne.	Must
U4	En tant qu'utilisateur, je veux consulter l'historique et le suivi de mes commandes pour savoir où elles en sont.	Must
U5	En tant qu'utilisateur, je veux modifier ou annuler une commande tant qu'elle n'est pas acceptée pour corriger une erreur.	Must
U6	En tant qu'utilisateur, je veux modifier mes informations personnelles pour les tenir à jour.	Must
U7	En tant qu'utilisateur, je veux déposer un avis après une commande terminée pour donner mon retour.	Must
U8	En tant qu'utilisateur, je veux recevoir des mails (bienvenue, confirmation, avis) pour être tenu informé.	Must

2.3 Employé

#	User story	Priorité
E1	En tant qu'employé, je veux gérer les menus, plats et horaires (CRUD) pour tenir le catalogue à jour.	Must
E2	En tant qu'employé, je veux faire évoluer le statut des commandes selon le workflow pour piloter les prestations.	Must
E3	En tant qu'employé, je veux filtrer les commandes par statut et par client pour m'organiser.	Should
E4	En tant qu'employé, je veux annuler ou modifier une commande après contact du client (motif + mode de contact obligatoires) pour tracer l'intervention.	Must

#	User story	Priorité
E5	En tant qu'employé, je veux modérer les avis (valider / refuser) pour contrôler ce qui est publié.	Must

2.4 Administrateur

#	User story	Priorité
A1	En tant qu'administrateur, je veux créer des comptes employés (avec mail de notification) pour gérer l'équipe.	Must
A2	En tant qu'administrateur, je veux désactiver un compte employé pour retirer un accès.	Must
A3	En tant qu'administrateur, je veux un tableau de bord des statistiques (commandes par menu, CA) alimenté par MongoDB pour piloter l'activité.	Must
A4	En tant qu'administrateur, je veux filtrer le CA par menu et par période pour analyser les ventes.	Should

2.5 Transversal (technique et qualité)

#	Tâche technique	Priorité
T1	Socle : initialisation Symfony, connexion BDD, schema.sql / seed.sql.	Must
T2	Design system (CSS custom) et intégration responsive des maquettes.	Must
T3	Sécurité : validation des entrées, protection OWASP, gestion des rôles, durcissement.	Must
T4	Accessibilité RGAA.	Must
T5	Déploiement en ligne + documentation.	Must
T6	Livrables documentaires (doc technique, charte, manuel, gestion de projet, copie).	Must
T7	Paiement en ligne réel, notifications SMS, application mobile native.	Won't

Le "Won't" liste ce qui a été volontairement écarté : hors du périmètre du sujet, il n'aurait fait qu'ajouter du risque sans rapporter de points.

3. Découpage en étapes

Le travail a été organisé en étapes thématiques cohérentes. Chaque étape regroupe des fonctionnalités liées et se termine par un état testable mergé dans `develop`. Les dates correspondent aux commits réels du dépôt.

Étape 0 - Socle technique (25-26 mai)

Mise en place de l'environnement, du dépôt Git et des fondations : projet Symfony, connexion MySQL, schéma de base de données (`schema.sql` , `seed.sql` , entités Doctrine), puis authentification (inscription, connexion, réinitialisation du mot de passe).

Branches : `feature/project-setup` , `feature/database-schema` , `feature/auth-system` .

Étape 1 - Design system et pages publiques (29 mai - 1er juin)

Création du design system (CSS custom issu des maquettes Figma), puis intégration de toutes les pages accessibles sans connexion : accueil, liste des menus avec filtres dynamiques, détail d'un menu, contact, pages légales.

Branches : `feature/design-system` , `feature/homepage` , `feature/menu-details` , `feature/contact-page` , `feature/legal-pages` .

Étape 2 - Parcours utilisateur (2 juin)

Coeur fonctionnel côté client : passage de commande avec calcul du prix et des frais de livraison, espace personnel (historique, suivi de commande horodaté, modification du profil, annulation), et dépôt d'avis après une commande terminée.

Branches : `feature/order-system` , `feature/user-dashboard` , `feature/user-reviews` .

Étape 3 - Espaces employé et administrateur (21-24 juin)

Back-office : CRUD des menus, plats et horaires, gestion des commandes avec le workflow de statuts, modération des avis, gestion des comptes employés par l'administrateur, et tableau de bord statistique alimenté par MongoDB.

Branches : `feature/employee-menu-management` , `feature/employee-order-management` , `feature/employee-review-moderation` , `feature/admin-user-management` , `feature/admin-stats-dashboard` .

Étape 4 - Déploiement, sécurité et accessibilité (25-28 juin)

Mise en production et durcissement : conteneurisation Docker et déploiement sur Render, audit de sécurité (throttling de connexion, en-têtes HTTP, cookies de session), mise à jour des dépendances (correctifs de vulnérabilités), intégration des images de démonstration, et audit d'accessibilité RGAA.

Branches : `feature/deployment` , `feature/security-audit` , `feature/security-dependencies` , `feature/menu-images` , `feature/homepage-hero-image` , `feature/accessibility-rgaa` .

Étape 5 - Documentation et livrables (à partir du 29 juin)

Production des livrables : documentation technique (avec diagrammes MCD, cas d'utilisation, séquence, déploiement), charte graphique, maquettes et wireframes, manuel d'utilisation, documentation de gestion de projet, et copie à rendre. Vérifications finales et rotation des secrets avant remise.

Cette dernière étape ne produit pas de code mais consolide tout ce qui sera présenté au jury.

4. Workflow Git

Le sujet impose un workflow de branches à trois niveaux, respecté sur tout le projet.

```
main      (production - ce qui est déployé)
├─ develop (intégration - branche stable de travail)
│   ├── feature/auth-system
│   ├── feature/order-system
│   └─ feature/... (une par fonctionnalité)
```

Règles suivies :

- Chaque fonctionnalité part d'une branche `feature/*` issue de `develop`.
- Une fois la fonctionnalité codée et testée en local, la branche est mergée dans `develop`.
- Quand `develop` est stable et déployable, il est mergé dans `main`. Le déploiement Render se déclenche automatiquement sur `main`.
- Les commits sont atomiques et rédigés en français, avec un préfixe de type conventionnel (`feat` , `fix`) et le nom de la fonctionnalité, par exemple `feat(order-system): formulaire de commande, calcul prix/livraison`.

Ce découpage garantit que `main` reste toujours déployable et que chaque branche a un périmètre clair. Il correspond exactement au découpage du backlog : une carte Trello, une branche, une fonctionnalité.

5. Organisation Trello

J'ai monté un tableau Trello public (lien fourni dans la copie à rendre) pour poser le backlog et donner une vue d'ensemble du projet. Comme je l'ai dit plus haut, je m'en suis moins servi au quotidien que du dépôt Git, mais il reprend bien le principe Kanban d'un flux de gauche à droite et il resitue l'organisation du travail.

Colonnes :

Colonne	Rôle
Backlog	Toutes les user stories et tâches techniques non encore planifiées.
À faire	Les cartes sélectionnées pour l'étape en cours.
En cours	La carte sur laquelle je travaille (branche Git ouverte).
En test	Fonctionnalité codée, en cours de vérification locale avant merge.
Terminé	Fonctionnalité mergée dans <code>develop</code> .

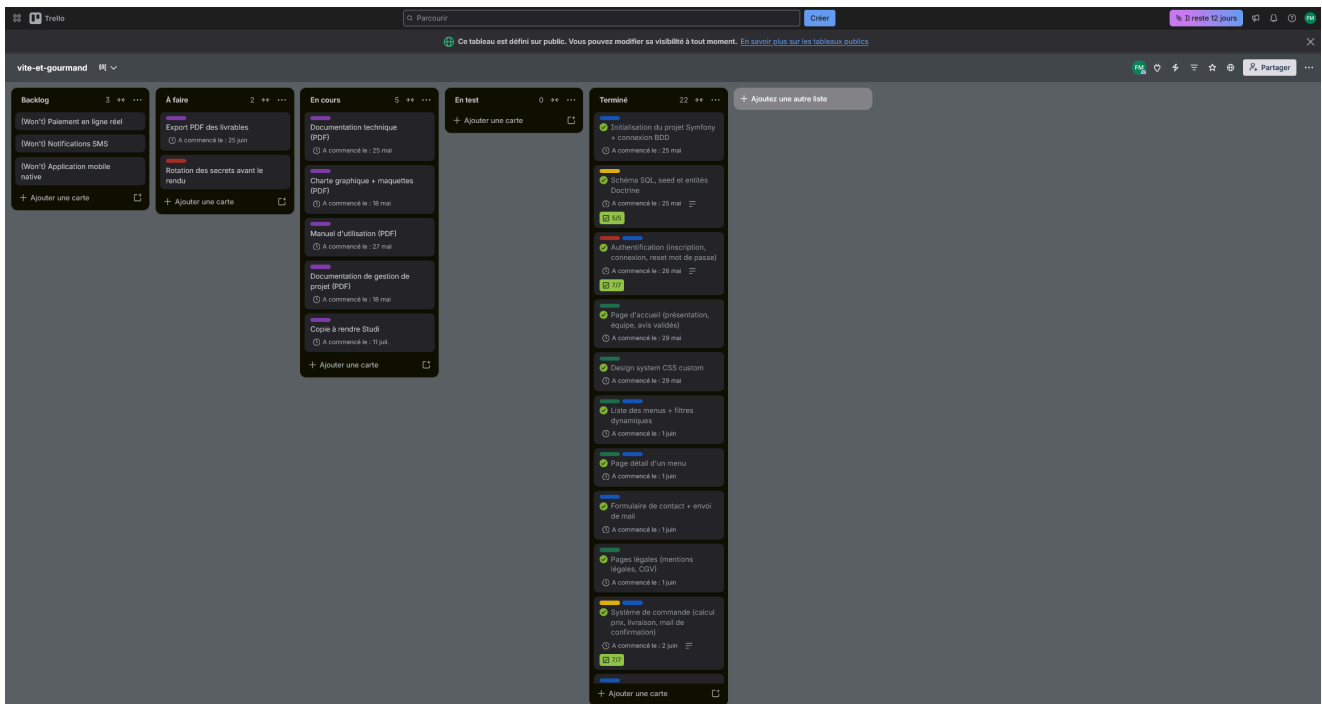
Labels par domaine : un code couleur distingue les natures de tâche pour lire le tableau d'un coup d'oeil.

- Front (intégration, UX, responsive)
- Back (contrôleurs, services, logique métier)
- BDD (schéma SQL, MongoDB, relations)
- Sécurité (auth, OWASP, RGPD)
- Doc (livrables, documentation)

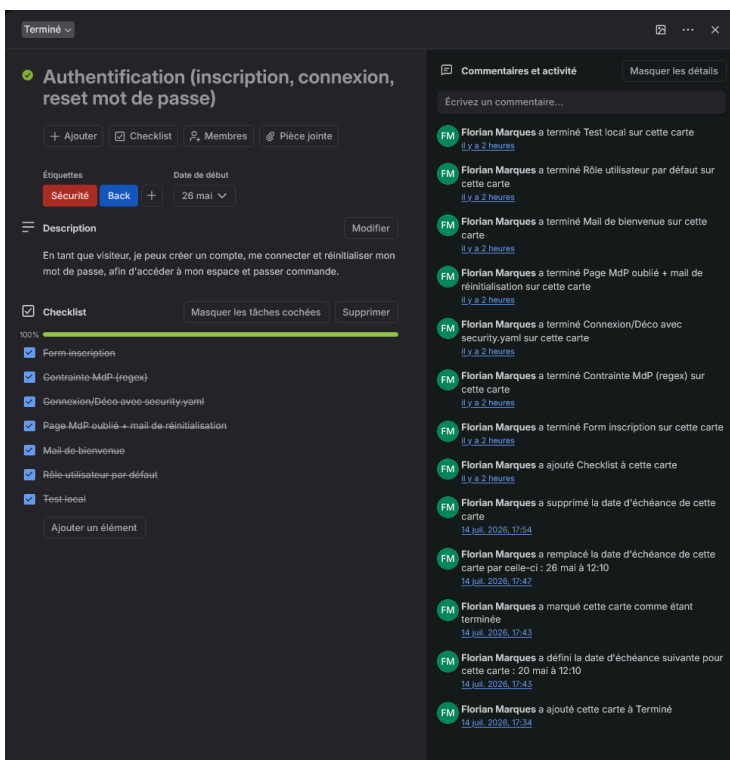
Contenu d'une carte : chaque carte correspond à une branche Git et porte le même nom que la fonctionnalité. Elle contient une description (rappel de la user story), une checklist des sous-tâches, et le

ou les labels de domaine. Cette correspondance carte / branche est ce qui relie le suivi de projet au dépôt de code.

Les captures ci-dessous montrent le tableau Trello : d'abord la vue d'ensemble des colonnes, puis le détail d'une carte.



Vue d'ensemble du tableau : les colonnes Backlog, À faire, En cours, En test et Terminé, avec les labels de domaine et les compteurs de checklist.



Détail de la carte "Authentification" : description sous forme de user story, checklist des sous-tâches et labels de domaine (Sécurité, Back).

6. Bilan et rétrospective

Ce qui a bien fonctionné :

- Le couple "une branche = une fonctionnalité" a rendu le projet lisible et facile à reprendre. Quand je revenais dessus, je repartais toujours d'un `develop` stable.
- La priorisation MoSCoW a évité la dispersion : aucune fonctionnalité "confort" n'a été codée avant que le périmètre obligatoire soit bouclé.
- La documentation produite au fil de l'eau (journal de bord des points techniques, mise à jour de l'état du projet) a beaucoup allégé la phase finale de rédaction des livrables.

Difficultés rencontrées et ajustements :

- Le déploiement m'a pris plus de temps que prévu. L'hébergeur que je visais au départ (Fly.io) demandait finalement une carte bancaire, j'ai donc basculé sur Render. Comme l'application et les bases (Aiven pour MySQL, Atlas pour MongoDB) sont hébergées séparément, ce changement n'a touché qu'un seul fichier de configuration.
- La partie statistiques MongoDB a nécessité une recherche spécifique (l'API HTTP d'Atlas ayant été arrêtée, il a fallu passer par l'extension PHP native et un mécanisme de synchronisation). C'est le point où j'ai dû chercher le plus.
- Mon rythme n'a pas été régulier. Des semaines de code bien remplies ont alterné avec des périodes où j'avancais moins sur le code parce que je cherchais et je testais des solutions. Le découpage en étapes indépendantes m'a permis d'absorber ça sans perdre la cohérence de l'ensemble.

Ce que je referais différemment : cadrer le choix de l'hébergeur plus tôt (vérifier les contraintes de compte avant de préparer la configuration de déploiement) et prévoir plus de temps pour la phase de documentation, que j'ai sous-estimée.